RSCL — Algorithm 3: Command Routing between HU and the Deterministic Mission Parser

Reconfigurable Space Computing Lab (RSCL), California State Polytechnic University, Pomona (Cal Poly Pomona) · Director: Prof. Mohamed El-Hadedy

Date: 2026-11-12 · Document type: formal algorithm specification of the implemented code (HU-MA'AT drone-fleet simulation demo)
Source files described (content hash and state at document build time):

`hu-telepresence-gazebo/festival/mission_intents.py`	`sha256 e109c5a9e89c`	2026-11-12 01:11 UTC	clean @ 05172be
`hu-telepresence-gazebo/festival/hu_live_server.py`	`sha256 2416370e93dc`	2026-11-11 06:38 UTC	clean @ 10ee643

1. Purpose and context

The frozen HU model knows nine primitive actions; it has no formation, yaw-turn-with-angle, spin or multi-aircraft travel vocabulary, and feeding such sentences to a closed-label classifier can map them onto a wrong primitive (for example a formation request classified as STOP). The router decides, deterministically and before any inference, whether an utterance (a) is refused, (b) becomes an operator-confirmed deterministic mission proposal, (c) is handed to the trained HU model, or (d) is shown to HU with execution forbidden. Every mission proposal is labelled as not originating from the trained model and requires explicit operator confirmation.

2. Inputs, outputs

Input: normalised utterance (after Algorithm 2 declined it). **Output of** `parse(text)`: one of `None` (route to HU), `{error}` (refuse), `{error, unrecognized: true}` (HU shown, never executed), or `{action, targets, params, label, reason}` with action ∈ {formation, spin, turn, takeoff, land, hover, move}. In the bridge, a proposal becomes a CONFIRM-band pending entry of kind `mission` (90 s, single use) and is sent on confirmation to `/api/mission/dispatch` with `operator_confirmed=true` and planner tag `HU-MISSION-V6`.

3. Algorithm

Algorithm 3.1 Mission router

Source: festival/mission_intents.py: parse, plain_primitive, distance_m

```
function ROUTE(text)
  s ← lower(text) stripped of " .!?", whitespace collapsed;  if s = "" then return error
  (names, problem) ← ADDRESSES(s)                          // Algorithm 4
  if s matches \b(then|after that|followed by|and then)\b then return error "compound"
  if ALT(s) and (TURN(s) or VALUE(s) or FORMATION(s)) then return error "compound"
  if FORMATION(s) then                                     // form, formation, shape, make, perform,
arrange, ...
    if problem then return error(problem)
    if |names| < 2 then return error "needs two aircraft"
    for alias in SHAPES sorted by decreasing length do   // whole-word alias match
      if alias occurs in s then return PROPOSE(formation, names, shape=SHAPES[alias])
    return error "unknown shape"
  if TURN(s) then                                          // turn, rotate, spin
    if "orbit" in s then return error;  if problem then return error(problem)
    if s has spin|full rotation|full turn|360|around yourself|in place then
      if an angle other than 360 is stated then return error
      return PROPOSE(spin, names, direction = −1 if right|clockwise else +1)
    deg ← stated "N degrees" (1–3 digits) else (180 if "around" in s else 90)
    if deg ∉ [5,180] then return error
    if count(left|counterclockwise|anticlockwise present, right|clockwise present) ≠ exactly one then
return error
    return PROPOSE(turn, names, degrees = ±deg)          // + = left (counter-clockwise)
```

```
  if ALT(s) then                                          // take off/launch/lift off, land/touch down, hover/hold/stop/freeze
    if problem then return error(problem)
    if number of distinct verb families present ≠ 1 then return error "conflicting verbs"
    if PLAIN-PRIMITIVE(s) then return None                // → trained HU model (Alg. 1)
    return PROPOSE(takeoff | land | hover, names)
  if MOVE(s) then
    if problem then return error(problem);  if names = ∅ then return error
    dirs ← {up, down, forward, back, left, right} present in s
    if |dirs| ≠ 1 then return error
    (m, err) ← DISTANCE-M(s);  if err then return error(err)
    return PROPOSE(move, names, dz | forward_m | right_m = ±m)   // >1 name ⇒ swarm travel (Alg. 8)
  if UNSUPPORTED(s) then return error "mission not implemented"   // orbit, patrol, scan, dance, flip, ...
  return {error, unrecognized: true}
function PLAIN-PRIMITIVE(s)
  b ← s with punctuation removed;  remove "except|excluding|but not <name>"
  remove names and filler {all, everyone, everybody, drone(s), the, fleet, swarm, five, and, please, now, ok, okay, hey, guys, team, you}
  return collapse(b) ∈ {take off, takeoff, launch, lift off, land, touch down, hover, hold position, stop}
function DISTANCE-M(s)
  V ← all matches of "<number> <unit>" with unit ∈ {cm, centimet(er|re)s, m, meter(s), metre(s)}
  if |V| ≠ 1 then return error "exactly one distance"
  m ← value × (0.01 if centimetres else 1)              // number words incl. "half" = 0.5
  if m ∉ [0.10, 2.0] then return error;  return round(m, 4)
```

Algorithm 3.2 Use in the HU bridge

Source: festival/hu_live_server.py: Service.command

```
procedure BRIDGE-ROUTE(u_norm)
  r ← ROUTE(u_norm);  unrecognized ← r ≠ None and r.unrecognized
  if unrecognized then r ← None                           // HU will be consulted, execution forbidden (Alg. 1, line 23)
  if r ≠ None then
    if r.error then return REFUSE(action = MISSION_UNSUPPORTED, reason = r.error)
    pending[id] ← (kind = mission, r.action, r.targets, r.params, now); return CONFIRM
  return HU-GUARD(u_norm, unrecognized)                   // Algorithm 1
```

4. Flowchart and state diagram

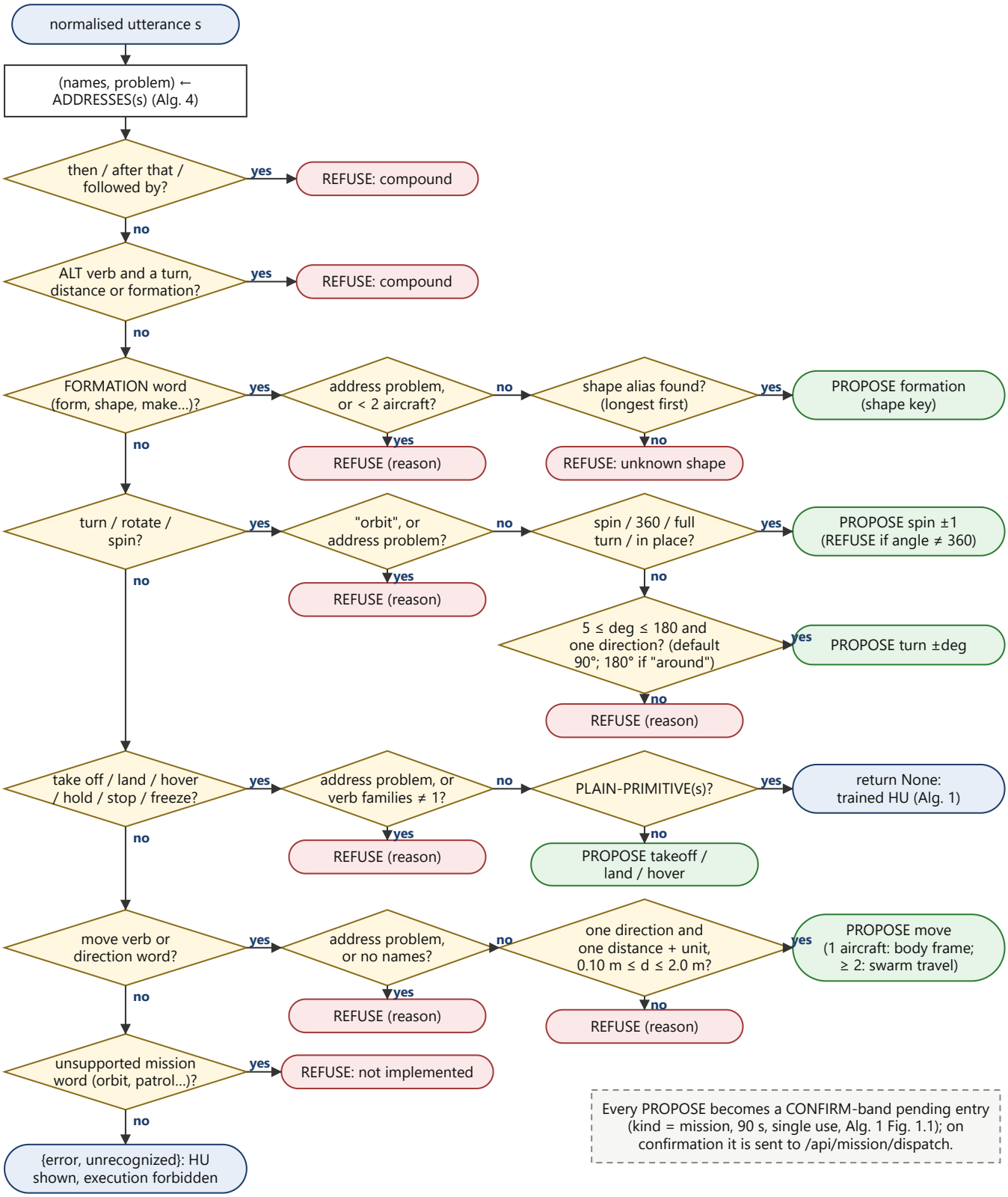


Figure 3.1. Flowchart of `mission_intents.parse` as called by the bridge. The cascade order is significant: the compound tests precede every branch, and FORMATION precedes TURN, ALT and MOVE. "Ra and Thoth, take off" is not compound (a name list); "take off and move forward 1 meter" is.

The router is stateless, so instead of a state machine its data flow is shown.

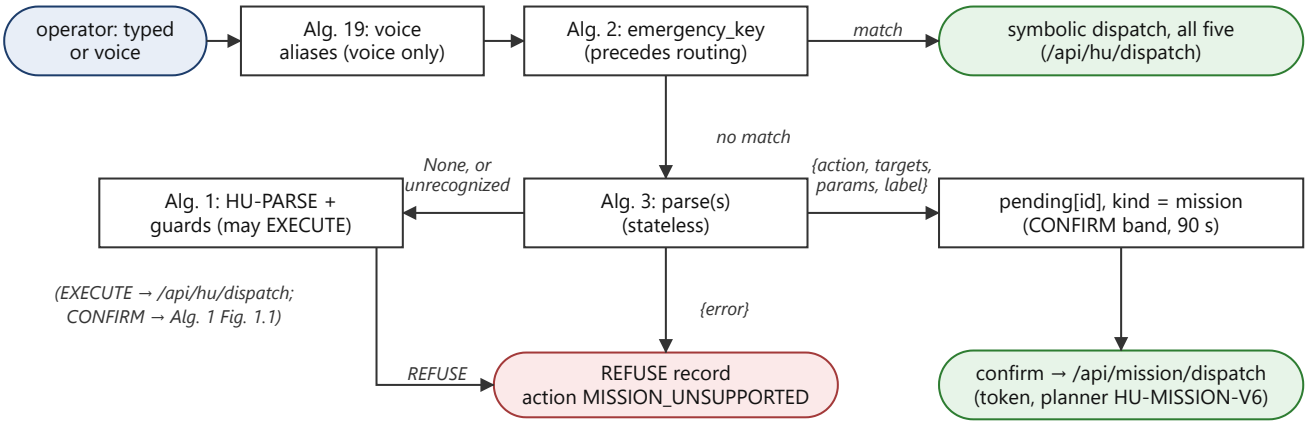


Figure 3.2. Data-flow view. The router is a pure function of the utterance: it keeps no state between calls and reads no telemetry. The only state downstream of it is the pending-confirmation table (Algorithm 1, Figure 1.1). Every outcome is appended to `logs/hu_real_decisions.jsonl`.

5. Parameters and vocabularies

Item	Value
Shape aliases	239 aliases → 106 shape keys in the current revision (the original table had 39 aliases → 25 keys; the extended catalogue adds letters, digits, constellations, theme, tactical, pentomino and transformed shapes; new single letters/digits are reachable only as "letter c", "c shape", "number 4", never as a bare letter, so the original short aliases v, x, t, l keep their meaning). Examples: *v shape, v formation, v → v*; *ring → circle*; *reverse v → inverted_v*; *communication relay → relay_chain*), longest alias tried first
Turn range / default	5..180°; default 90° (180° with "around")
Move range	0.10..2.0 m (±1e-8), exactly one distance with unit
Compound markers	*then, after that, followed by, and then*; and any altitude verb combined with a turn, a distance or a formation word
Confirmation	always required for proposals (band CONFIRM), 90 s lifetime

6. Complexity

A fixed cascade of regular-expression searches; the shape scan is O(|SHAPES|·|s|). Overall O(|s|) with a constant of a few dozen patterns. Measured routing latency is recorded per decision (`latency_ms`); no aggregate here.

7. Properties

Guaranteed

- **Fail-closed:** every branch either refuses with a reason, proposes a single action requiring operator confirmation, or delegates to HU; there is no default action.
- **Compound refusal:** sequential markers and verb combinations are refused instead of silently executing one part ("take off and move forward 1 meter").
- Formation, turn, spin and multi-aircraft move vocabulary never reaches the closed-label model.
- A list of names joined by *and* or commas is an address, not a compound ("Ra and Thoth, take off" reaches HU as a plain primitive); two *actions* joined by *and* ("land and turn left") are refused.
- Sentences with no recognised flight structure are shown to the model for transparency but cannot execute (enforced in Algorithm 1).

- Attribution: proposals carry the label "Deterministic simulator mission route ... not trained HU"; decision records carry `decision_source`.

Not guaranteed / limitations

- Coverage is lexical. Words such as *up*, *back*, *right* trigger the MOVE branch even in non-motion prose; such cases then fail on direction/distance requirements (refusal, not execution).
- A plain primitive with an *unaddressed* verb ("land") returns the address error from Algorithm 4 rather than reaching HU, because `problem` is checked before PLAIN-PRIMITIVE.
- Spoken number words are limited to the WORDS table (e.g. no "twenty five"); the turn angle requires digits.
- Routing does not check feasibility; feasibility is decided by the supervisor (Algorithms 5–10) after confirmation.

8. Testing and measured results

- `acceptance/tests/test_compound_commands.py` (refused compounds; single actions unaffected), `test_missions.py` (e.g. `test_v_intercept_does_not_call_hu`, `test_unsupported_refused_before_hu`, `test_turn_target`, `test_spin_360`, `test_shape_catalogue`), `test_v9.py` (`test_meaningful_mission_coverage`, `test_reject_unknown_ambiguous_conflicting_and_unsafe`, `test_all_form_does_not_call_hu`), `test_strategist.py::test_every_phrase_parses_as_a_formation`; `acceptance/formation_geometry_check.py` checks that each shape key is reachable by a phrase.
- Recorded: offline suite 44 tests OK and live non-actuating command matrix 28/28 PASS in `run_20261111T052914Z`. That run predates the latest change to `mission_intents.py` (introduction of `plain_primitive`, which now sends e.g. "All, land" to HU instead of a mission proposal); the current routing has **not been re-measured here** live.

9. Comparison with alternatives and related work

Mapping an utterance to an executable action is semantic parsing. The router is a hand-written, regular-expression cascade; the alternatives below are learned or grammar-based parsers and language-model tool calling.

- **Learned grammar-based semantic parsing** [1]: a probabilistic combinatory categorial grammar is induced from sentences paired with logical forms; outputs are compositional (sequences, conjunctions, nested arguments).
- **Grounded command understanding for robots** [2]: Generalized Grounding Graphs build a probabilistic model from the command's structure and ground phrases to objects, places and paths in the environment.
- **Constrained decoding with a language model** [3]: the model paraphrases input into a controlled canonical sublanguage under a grammar constraint, which maps deterministically to a meaning representation; only grammatical outputs can be produced.
- **Language-model tool / function calling** [4, 5]: a model selects and fills API calls; [5] applies prompting with a high-level function library to robots including aerial navigation in simulation.

Approach	Coverage of phrasing	Composition	Determinism / auditability	Failure mode
CCG semantic parser [1]	learned from data	yes	deterministic given the model; grammar inspectable	wrong parse with high score
Grounding graphs [2]	learned, corpus-based	yes, with spatial grounding	probabilistic	wrong grounding
Constrained LM decoding [3]	broad (paraphrase)	as the target grammar allows	output always well-formed; choice not deterministic across models	well-formed but wrong meaning

LLM tool calling [4, 5]	very broad	yes, multi-step	sampling-dependent; needs external guards	plausible but wrong call or argument
This router	fixed vocabulary (e.g. 239 shape aliases)	none: one action per utterance, compounds refused	pure function; every branch inspectable; refuses by default	refusal of valid phrasing

Where the router is weaker. Its coverage is lexical and small: paraphrases, synonyms outside the tables, spelled numbers beyond the WORDS table and any sequencing are refused, and no learning or measurement of coverage on real operator speech exists. It cannot express multi-step missions, which [1], [3] and [4, 5] can. **Where it differs.** It never invents an aircraft, an action or a parameter, its latency is a regular-expression scan, it runs without network or model, and its output is always either a refusal or a single proposal that the operator must confirm. A learned parser placed behind the same confirmation step and supervisor checks would be a compatible extension.

10. Relation to other algorithms

Runs after Algorithm 2 and before Algorithm 1. Uses Algorithm 4 for addressing. Formation proposals are executed by Algorithm 5, multi-aircraft moves by Algorithm 8, take-offs by Algorithm 6. SwarmChat (Algorithm 12) calls `parse` only as a read-only preview.

11. References

1. L. S. Zettlemoyer and M. Collins, "Learning to Map Sentences to Logical Form: Structured Classification with Probabilistic Categorial Grammars," *UAI* 2005, pp. 658–666. `https://arxiv.org/abs/1207.1420`
2. S. Tellex, T. Kollar, S. Dickerson, M. R. Walter, A. G. Banerjee, S. Teller and N. Roy, "Understanding Natural Language Commands for Robotic Navigation and Mobile Manipulation," *AAAI* 2011, pp. 1507–1514. `https://doi.org/10.1609/aaai.v25i1.7979`
3. R. Shin, C. H. Lin, S. Thomson, C. Chen, S. Roy, E. A. Platanios, A. Pauls, D. Klein, J. Eisner and B. Van Durme, "Constrained Language Models Yield Few-Shot Semantic Parsers," *EMNLP* 2021, pp. 7699–7715. `https://doi.org/10.18653/v1/2021.emnlp-main.608`
4. T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda and T. Scialom, "Toolformer: Language Models Can Teach Themselves to Use Tools," *NeurIPS* 2023. `https://arxiv.org/abs/2302.04761`
5. S. Vemprala, R. Bonatti, A. Bucker and A. Kapoor, "ChatGPT for Robotics: Design Principles and Model Abilities," *IEEE Access*, 2024; preprint arXiv:2306.17582. `https://arxiv.org/abs/2306.17582`

This specification describes the behaviour of the source revision identified above. Line-level references use `file:function`. Where the code and its own comments disagree, the code is described. Simulation-only system: no statement here is a claim about flight hardware.